Lab 2: RAW Image Processing

Due Wednesday, February 11, 11:59pm

In this lab, you will build your own version of a basic image processing pipeline. You will use this to turn RAW images captured with the Canon EOS R10 cameras provided in class into images that can be displayed on a computer monitor. As we discussed in class, RAW images do not look like standard images before first undergoing a "development" process.[1] The final result can vary greatly, depending on the choices you make in your implementation of the image processing pipeline.

Allowed libraries. You may use `numpy` for array math, `scipy` for interpolation and linear algebra, `matplotlib` for visualization, and `imageio/skimage` for saving images. You may use `rawpy` *only* to read the RAW file and access metadata (black level, white level, Bayer pattern, camera white balance multipliers, etc.).

Not allowed. Calling `raw.postprocess()` (or any other library that performs demosaicing, color correction, tone mapping, etc.) to produce your final outputs. You may use `raw.postprocess()` only in the comparison section.

There is a "Hints and Information" section at the end of this document that is likely to help. We strongly recommend that you read that section in full before you start to work on the lab.

1 Understanding RAW Images

Digital cameras contain image sensors (typically CMOS) that measure light intensity at millions of discrete locations called *pixels*. Most consumer cameras use a *Bayer filter* placed over the sensor, so each pixel only measures one color channel (red, green, or blue). The resulting "RAW" data is a single-channel mosaic that must be processed before it can be displayed.

Problem 1.1: Why does the Bayer pattern contain twice as many green pixels as red or blue?

Human vision is most sensitive to green light, so having more green samples helps improve perceived sharpness and reduce noise in the luminance channel

Problem 1.2: If a sensor has 6000 × 4000 Bayer-mosaiced pixels, what is the resolution of the final RGB image after demosaicing?

The size would stay the same (6000 x 4000)

When you shoot in JPEG mode, the camera applies its internal image processing pipeline (linearization, white balance, demosaicing, color correction, tone mapping, sharpening, compression) and discards the original sensor data. When you shoot in RAW mode, the camera saves the unprocessed sensor measurements along with metadata (exposure settings, white balance coefficients, etc.).

Problem 1.3: Name two advantages of shooting in RAW mode instead of JPEG.

1. You can recover details that would be clipped in a JPEG
2. You can change the color temperature after capture without a loss in quality

[1] The term "develop" comes from the analogy with the process of developing film into a photograph. As discussed in class, we often refer to this process as "rendering" the RAW images.

2 Developing RAW Images

Capture at least **three** different scenes with the Canon R10:

- One outdoor (daylight) scene.
- One indoor (tungsten/LED) scene.
- One scene of your choice (try something interesting!).

Set the camera to **RAW+JPEG** mode so you have the camera's developed JPEG for comparison. The camera produces `.CR3` files containing the Bayer-mosaiced sensor measurements.

2.1 Reading and Linearization

Use `rawpy` to load your `.CR3` file. We will be using `raw.raw_image_visible` for the mosaic data (this excludes sensor border pixels). The data will be in the form of a `numpy` 2D-array of unsigned integers.

```
import rawpy
import numpy as np
import matplotlib.pyplot as plt
```

Problem 2.1.1: Load one of your RAW images and report:

(a) The width and height of the raw mosaic. How does this compare to the sensor resolution?

(b) The data type (`dtype`) and number of bits per pixel.

(c) The `white_level` value.

(d) The `black_level_per_channel` values.

(e) The camera white balance multipliers (`raw.camera_whitebalance`).

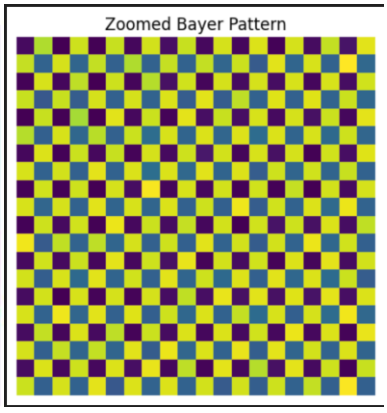
Width: 6000, Height: 4000 dtype: uint16 bits/pixel: 16

white level: 16383 black_level_per_channel: [511, 511, 511, 511]

White balance multipliers: [1511.0, 1024.0, 1777.0, 1024.0]

Convert the image into a double-precision (`float64`) array. The raw values are not yet linear—they have an offset due to dark noise and saturated pixels from over-exposure. Convert to a linear array in $[0, 1]$ by mapping the black level to 0 and white level to 1, then clipping.

Problem 2.1.2: Implement linearization. Display the linearized image (you may want to boost the brightness for visibility). Also display a zoomed-in crop that clearly shows the Bayer mosaic pattern.



2.2 Bayer Pattern and White Balancing

Most cameras use the Bayer pattern to capture color. The top-left 2×2 square can correspond to any of four possible configurations, as shown in Figure 1.

Problem 2.2.1: Determine which Bayer pattern applies to your camera. Explain how you identified it.

The RGGB pattern applies because from the observed Zoomed Bayer Pattern, it looks approximately that it is RGGB (just a little shifted)

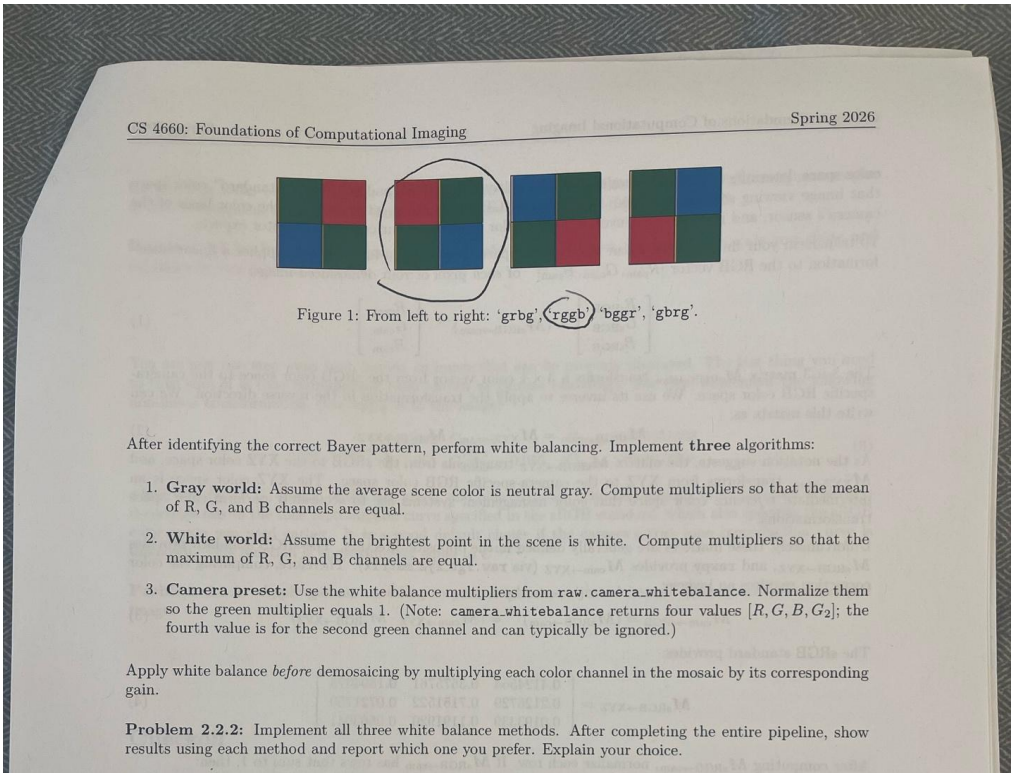


Figure 1: From left to right: 'grbg', 'rggb' 'bggr', 'gbrg'.

After identifying the correct Bayer pattern, perform white balancing. Implement **three** algorithms:

1. **Gray world:** Assume the average scene color is neutral gray. Compute multipliers so that the mean of R, G, and B channels are equal.
2. **White world:** Assume the brightest point in the scene is white. Compute multipliers so that the maximum of R, G, and B channels are equal.
3. **Camera preset:** Use the white balance multipliers from `raw.camera_whitebalance`. Normalize them so the green multiplier equals 1. (Note: `camera_whitebalance` returns four values $[R, G, B, G_2]$; the fourth value is for the second green channel and can typically be ignored.)

Apply white balance *before* demosaicing by multiplying each color channel in the mosaic by its corresponding gain.

Problem 2.2.2: Implement all three white balance methods. After completing the entire pipeline, show results using each method and report which one you prefer. Explain your choice.

I personally prefer camera preset, but between camera preset and gray world, there aren't that many noticeable differences (white world is obviously inferior because of its green hue)

Gray World

White World

Camera Preset

2.3 Demosaicing

Once white balancing is done, demosaic the image using bilinear interpolation. Use `scipy.interpolate.RectBivariateSpline` with `kx=1,ky=1` for bilinear interpolation. This function lets you define a spline on the subsampled grid and then evaluate it at arbitrary positions—including the half-integer positions needed to upsample by 2x.

Problem 2.3.1: Demosaic your white-balanced mosaic using bilinear interpolation. Report the output image dimensions. Display the result.

(Using the **camera preset**), the image dimensions are the same (4000*6000 image)

color space determined by the camera's spectral sensitivity functions, and not in the "standard" color space that image viewing software expects. Each pixel's RGB values represent a vector in the color basis of the camera's sensor, and needs to be converted to the color basis that your computer monitor expects.

To transform your image to the linear sRGB color space, implement a function that applies a linear transformation to the RGB vector $[R_{\text{cam}}, G_{\text{cam}}, B_{\text{cam}}]^\top$ of *each* pixel of your demosaiced image:

$$\begin{bmatrix} R_{\text{sRGB}} \\ G_{\text{sRGB}} \\ B_{\text{sRGB}} \end{bmatrix} = (\boldsymbol{M}_{\text{sRGB}\to\text{cam}})^{-1} \cdot \begin{bmatrix} R_{\text{cam}} \\ G_{\text{cam}} \\ B_{\text{cam}} \end{bmatrix}. \tag{1}$$

The 3×3 matrix $\boldsymbol{M}_{\text{sRGB}\to\text{cam}}$ transforms a 3×1 color vector from the sRGB color space to the camera-specific RGB color space. We use its inverse to apply the transformation in the reverse direction. We can write this matrix as:

$$\boldsymbol{M}_{\text{sRGB}\to\text{cam}} = \boldsymbol{M}_{\text{XYZ}\to\text{cam}} \cdot \boldsymbol{M}_{\text{sRGB}\to\text{XYZ}}. \tag{2}$$

As the notation suggests, the matrix $\boldsymbol{M}_{\text{sRGB}\to\text{XYZ}}$ transforms from the sRGB to the XYZ color space, and $\boldsymbol{M}_{\text{XYZ}\to\text{cam}}$ transforms from XYZ to the camera-specific RGB color space. The XYZ color space is an intermediate *reference* color space that color management systems use to accurately perform color space transformations.

Unfortunately, these matrices are generally defined in the opposite direction. The sRGB standard provides $\boldsymbol{M}_{\text{sRGB}\to\text{XYZ}}$, and `rawpy` provides $\boldsymbol{M}_{\text{cam}\to\text{XYZ}}$ (via `raw.rgb_xyz_matrix`). Therefore, computing the color correction requires an inverse:

$$\boldsymbol{M}_{\text{cam}\to\text{sRGB}} = (\boldsymbol{M}_{\text{sRGB}\to\text{cam}})^{-1} = (\boldsymbol{M}_{\text{cam}\to\text{XYZ}} \cdot \boldsymbol{M}_{\text{sRGB}\to\text{XYZ}})^{-1}. \tag{3}$$

The sRGB standard provides:

$$\boldsymbol{M}_{\text{sRGB}\to\text{XYZ}} = \begin{bmatrix} 0.4124564 & 0.3575761 & 0.1804375 \\ 0.2126729 & 0.7151522 & 0.0721750 \\ 0.0193339 & 0.1191920 & 0.9503041 \end{bmatrix}. \tag{4}$$

After computing $\boldsymbol{M}_{\text{sRGB}\to\text{cam}}$, normalize each row. If $\boldsymbol{M}_{\text{sRGB}\to\text{cam}}$ has rows that sum to 1, then:

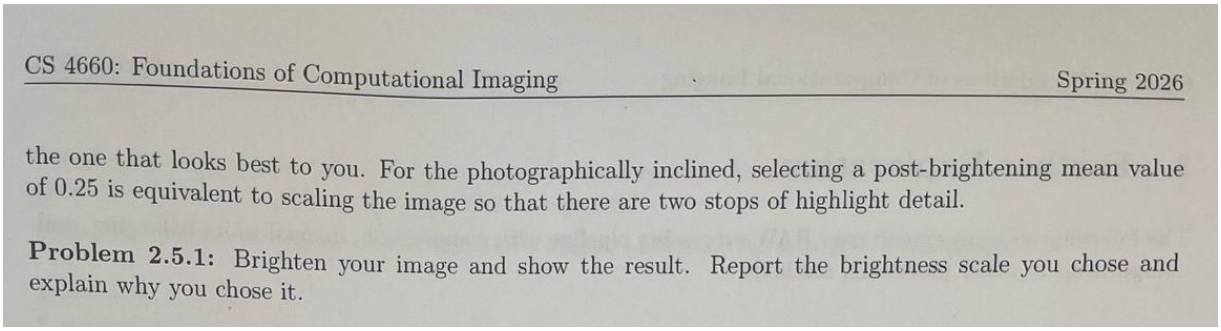
$$\boldsymbol{M}_{\text{sRGB}\to\text{cam}} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}_{\text{sRGB}} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}_{\text{cam}}, \tag{5}$$

which means the inverse $\boldsymbol{M}_{\text{cam}\to\text{sRGB}}$ also maps white to white, preserving your white balance.

Problem 2.4.1: Use `raw.rgb_xyz_matrix` to compute the color correction matrix. Apply it to your demosaiced image. Show before and after images. Describe the most noticeable color changes.

Demosaiced Image

It seems to have gotten much bluer/colorful, and less “gray” (uses the camera preset white balancing)



CS 4660: Foundations of Computational Imaging Spring 2026

the one that looks best to you. For the photographically inclined, selecting a post-brightening mean value of 0.25 is equivalent to scaling the image so that there are two stops of highlight detail.

Problem 2.5.1: Brighten your image and show the result. Report the brightness scale you chose and explain why you chose it.

To me, I basically just chose whatever the default was but a little brighter, because the final image looked approximately as bright as real life

You are now one step away from having an image that can be properly displayed. The last thing you need to take care of is tone reproduction (also known as gamma encoding). For this, implement the following non-linear transformation, then apply it to the image:

$$C_{\text{non-linear}} = \begin{cases} 12.92 \cdot C_{\text{linear}}, & C_{\text{linear}} \leq 0.0031308, \\ 1.055 \cdot C_{\text{linear}}^{1/2.4} - 0.055, & C_{\text{linear}} > 0.0031308, \end{cases} \tag{6}$$

where $C \in \{R, G, B\}$ is each of the red, green, and blue channels. This odd-looking function is not arbitrary: it corresponds to the tone reproduction curve specified in the sRGB standard, which also specifies the sRGB color space you used earlier. It is a good default choice if the camera's true tone reproduction curve is unknown. This function is approximately equal to $C^{1/2.2}$.

Problem 2.5.2: Apply gamma encoding to your brightness-adjusted image. Show your final developed image.

After Color Correction (sRGB)

3 Further Exploration

The following sections extend your RAW processing pipeline with compression, manual white balancing, and comparisons to other processing methods.

3.1 Compression

Finally, it is time to store the image, either with or without compression. Use `imageio.imwrite` to store the image in `.png` format (lossless), and also in `.jpg` format with the `quality` parameter set to 95. This setting determines the amount of compression.

Problem 3.1.1: Save your image as PNG and JPEG (quality 95). Can you tell the difference between the two files? The compression ratio is the ratio between the size of the uncompressed file (in bytes) and the size of the compressed file (in bytes). What is the compression ratio? By changing the JPEG quality settings, determine the lowest setting for which the compressed image is indistinguishable from the original. What is the compression ratio at that setting?

2PNG

1JPG 95

PNG Size: 25846226 bytes

JPEG Size (Q95): 3928240 bytes

Compression Ratio: 6.58

The lowest compression setting we can set with the images staying indistinguishable occurs around 25.

PNG Size: 25846226 bytes

JPEG Size (Q95): 647759 bytes

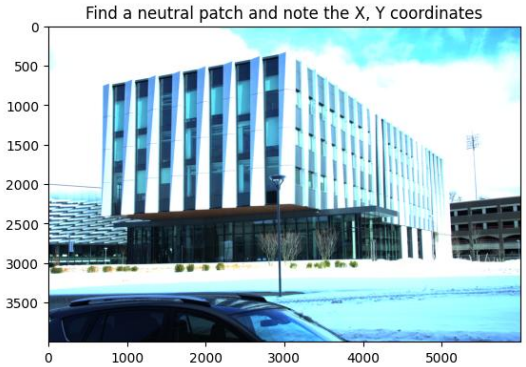
Compression Ratio: 39.90

3- JPG 25

3.2 Manual White Balancing

One way to do manual white balancing is by: (1) selecting some patch in the scene that you expect to be white; and (2) normalizing all three channels using weights that make the red, green, and blue channel values of this patch be equal. (See `matplotlib.pyplot.ginput` for interactively recording image coordinates.) Apply these gains to the original linearized mosaic, then re-run the rest of your pipeline.

Problem 3.2.1: Implement manual white balancing and experiment with different patches in the scene. Show the corresponding results, and discuss which patches work best.



The patches that worked best were the ones that were closest to the actual white color in the origional image, as they would lead to the least intrusive filter color.

3.3 Comparisons

Problem 3.3.1: Use `raw.postprocess()` to develop the same RAW image. Compare your developed image with the `rawpy` result and with the camera's JPEG. Show side-by-side comparisons, explain any differences between them, and report which of the images you like best.

All 3 images have pretty similar quality and share the same details. However, All three of them appear to have a different level of exposure. From darkest to lightest. We have the manually developed photo, the postprocess() photo, and then finally the camera's jpg. I personally like the postprocess() photo the best. I believe that it stikes the right balance with the amount of light. Nothing appears too bright or dark.

Hints and Information

Displaying intermediate results. Until the final gamma encoding step, all your processing is in linear space. Linear images appear very dark when displayed directly. To visualize intermediate results, multiply by a constant (e.g., 5) and clip:

```
plt.imshow(np.clip(img_intermediate * 5, 0, 1))
plt.show()
```

Gamma encoding should only be applied once, at the very end. Additionally, colors will be very off (strong green hue) until after white balancing. This is expected!

Indexing Bayer channels. Use NumPy slice syntax to extract the four Bayer sub-images:

```
# For RGGB pattern:
R = mosaic[0::2, 0::2] # top-left pixels
G1 = mosaic[0::2, 1::2] # top-right pixels
G2 = mosaic[1::2, 0::2] # bottom-left pixels
B = mosaic[1::2, 1::2] # bottom-right pixels

# Combine into quarter-resolution RGB for visualization:
rgb_quarter = np.stack([R, (G1+G2)/2, B], axis=-1)
```

Bilinear interpolation for demosaicing. `RectBivariateSpline` lets you define a spline on the sub-sampled grid coordinates `(0,1,...,h-1)` and evaluate at half-integer positions `(0,0.5,1,1.5,...)` to upsample by 2x:

```
from scipy.interpolate import RectBivariateSpline

h, w = R.shape
spline = RectBivariateSpline(np.arange(h), np.arange(w), R, kx=1, ky=1)
R_full = spline(np.arange(2*h) / 2, np.arange(2*w) / 2)
```

This preserves the original measured values at integer positions and interpolates only where data is missing.

Applying color matrix. To apply a 3×3 matrix to every pixel efficiently, you can use either `np.einsum` or matrix multiplication with a transpose. Both produce the same result:

```
# img has shape (H, W, 3), M has shape (3, 3)
img_corrected = np.einsum('ij,...j', M, img) # M @ pixel for each pixel
# Or equivalently:
img_corrected = img @ M.T
```

Debugging colors. If your colors look completely wrong (strong magenta or green cast), check:

- Is your Bayer pattern interpretation correct? Try all four patterns.
- Are you applying white balance to the correct channels?
- Did you normalize the color correction matrix rows to sum to 1?

Memory and performance. Convert to `float32` or `float64` early to avoid integer overflow. For faster iteration, work on a cropped region first, then process the full image once your pipeline works.

Submission

Submit the following on Gradescope:

- A single PDF with answers to each problem, including all requested figures and comparisons. Submit this to "Lab 2 Writeup."
- A `.zip` file containing all Python code. Include a `README.md` explaining how to run your code. Submit this to "Lab 2 Code."